DEPARTMENT OF AVIONICS ENGINEERING

SUBJECT : **Digital Systems Microprocessors s Microcontrollers Lab**

SUBJECT CODE : **308432**

LAB NO : **03**

TITLE : **Hardware Timers & Internal Interrupts**

SUBMITTED TO : **Engr M. Ali Ahmed**

SEMSTER : **4th**

SECTION : **A**

Marks Obtained

	Group Member 1	Group Member 2	Group member 3
NAME	Sardar Ahmed Xaremann	M.Umer Khan	M.Rahid
REGISTRATION ID	240701019	240701033	240701037
LAB REPORT			
PERFORMANCE			
TOTAL MARKS			

DEADLINE:	
DATE OF SUBMISSION:	

WORK DIVISION TABLE

Project Component	Responsible Member	Primary Tasks
Code Development	**Xaremann**	Implementing Timer0/Timer1 ISR, multi-flag logic, and 16-bit preload math.
Simulation & Testing	**Umer**	Proteus circuit setup with LEDs on RB0, RB2, RD0, and Oscilloscope verification.
Datasheet Analysis	**Xaremann & Umer**	Analyzing OPTION_REG, INTCON, and T1CON for prescaler and interrupt bits.
Report Drafting	**Rahid**	Documenting the methodology, results, and formatting the final submission.

Introduction

In embedded systems design, the CPU often needs to manage multiple time-sensitive tasks simultaneously. Traditional software delays (like `__delay_ms`) are "blocking," meaning they force the CPU to execute useless "NOP" instructions, effectively blinding it to other tasks while waiting. **Hardware Timers** solve this by acting as independent digital counters that run in the background of the main silicon.

This lab explores **Internal Interrupts**, where a timer "rings an alarm" upon reaching a specific count, triggering an **Interrupt Service Routine (ISR)**. This allows for **Embedded Multi-tasking**, where a high-priority background task (like toggling an LED every 1 or 2 seconds) can occur without being affected by the "busy-wait" loops of the foreground code.

Objectives

The objectives for this lab report are as follows:

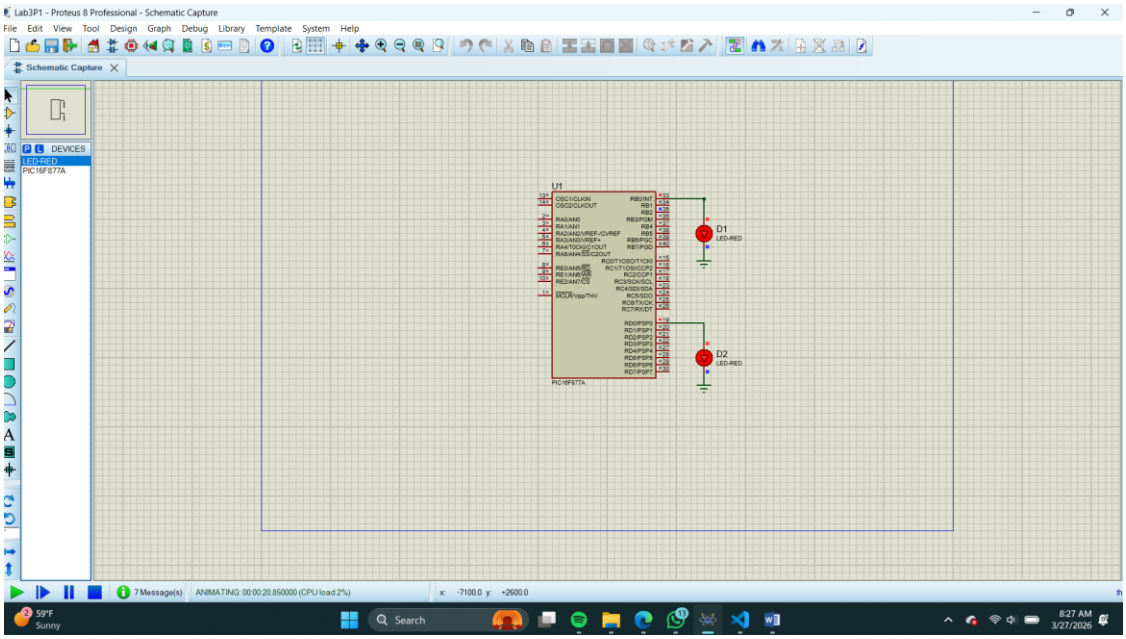
- **Concurrent Execution:** To transition from blocking software delays to hardware-timed execution.
- **Timer Resolution:** To implement precise time intervals using both 8-bit (**Timer0**) and 16-bit (**Timer1**) architectures.

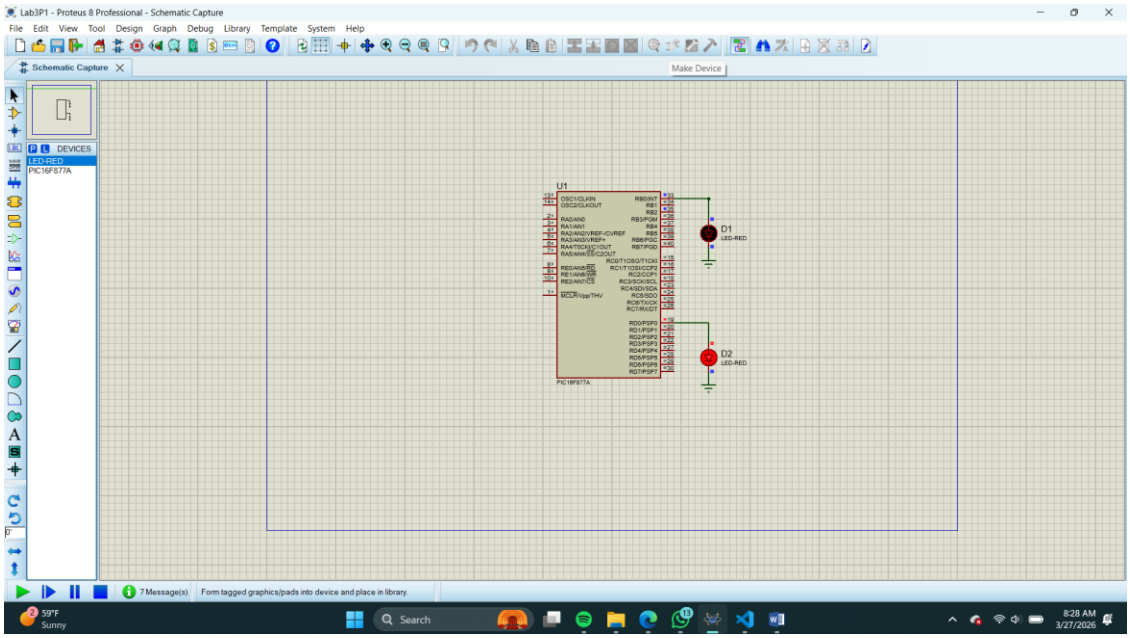
- **Embedded Multi-tasking:** To utilize Internal Interrupts to achieve background task execution while the main loop remains free.
- **Register Management:** To master the configuration of control registers such as `OPTION_REG`, `INTCON`, and `T1CON`.
- **Multi-Interrupt Management:** To design a "switchboard" logic ISR that independently handles multiple interrupt flags (`TMR0IF` and `TMR1IF`).
- **AI Integration:** To utilize AI-assisted development for calculating 16-bit preload values and structuring multi-flag ISR code.

Tasks:

Task #1:

Proteus Simulation:





Code:

```c
#include <xc.h>

// Configuration Bits
#pragma config FOSC = XT      // Oscillator Selection (4MHz Crystal)
#pragma config WDTE = OFF     // Watchdog Timer Disabled
#pragma config PWRTE = OFF    // Power-up Timer Disabled
#pragma config BOREN = ON     // Brown-out Reset Enabled
#pragma config LVP = OFF      // Low-Voltage Programming Disabled

#define _XTAL_FREQ 4000000

// Global Variables
unsigned int t0_overflow_count = 0;
unsigned int t1_overflow_count = 0;

// Interrupt Service Routine
void __interrupt() ISR(void) {
    // Handle Timer0 (1 Second Toggle on RB0)
    if (INTCONbits.TMR0IF) {
        TMR0 = 61;               // Reload for 50ms [cite: 233, 342]
        t0_overflow_count++;
        if (t0_overflow_count >= 20) {
            PORTBbits.RB0 = ~PORTBbits.RB0; // Toggle LED 1
            t0_overflow_count = 0;
        }
        INTCONbits.TMR0IF = 0;   // Clear flag [cite: 71, 335]
    }

    // Handle Timer1 (2 Second Toggle on RB2)
    if (PIR1bits.TMR1IF) {
        TMR1H = 0x0B;            // Preload High Byte (3036) [cite: 340]
        TMR1L = 0xDC;            // Preload Low Byte
        t1_overflow_count++;
        if (t1_overflow_count >= 4) {
            PORTBbits.RB2 = ~PORTBbits.RB2; // Toggle LED 3
            t1_overflow_count = 0;
        }
        PIR1bits.TMR1IF = 0;     // Clear flag [cite: 314, 335]
    }
}

void main(void) {
    // I/O Initialization
    TRISB0 = 0; TRISB2 = 0;      // Set RB0 and RB2 as outputs [cite: 260]
    TRISD0 = 0;                  // Set RD0 as output [cite: 262]
    PORTB = 0x00; PORTD = 0x00;
```

```
    // Timer0 Configuration
    OPTION_REGbits.T0CS = 0;      // Internal clock [cite: 63]
    OPTION_REGbits.PSA = 0;       // Assign prescaler to Timer0 [cite: 64]
    OPTION_REGbits.PS = 0b111;    // 1:256 Prescaler [cite: 65]
    TMR0 = 61;

    // Timer1 Configuration
    T1CONbits.T1CKPS = 0b11;      // 1:8 Prescale value
    T1CONbits.TMR1CS = 0;         // Internal clock
    TMR1H = 0x0B; TMR1L = 0xDC;
    T1CONbits.TMR1ON = 1;         // Turn Timer1 ON

    // Interrupt Configuration
    INTCONbits.TMR0IE = 1;        // Enable Timer0 Interrupt [cite: 70]
    PIE1bits.TMR1IE = 1;          // Enable Timer1 Interrupt [cite: 326]
    INTCONbits.PEIE = 1;          // Enable Peripheral Interrupts [cite: 325]
    INTCONbits.GIE = 1;           // Enable Global Interrupts [cite: 323]

    while(1) {
        // Foreground Task: Fast Blink [cite: 80, 82]
        PORTDbits.RD0 = 1;
        __delay_ms(100);
        PORTDbits.RD0 = 0;
        __delay_ms(100);
    }
}
```

Explanation:

1. Configuration & Global Variables

- **Oscillator & Frequency:** The code sets the oscillator to **XT** for a 4 MHz crystal. This establishes an internal **Instruction Clock (FCY)** of 1 MHz (FOSC/4), meaning each instruction "tick" takes exactly **1 mus**.
- **Software Counters:** Two unsigned int variables (t0_overflow_count and t1_overflow_count) act as extended counters. Since hardware timers have limits (8-bit or 16-bit), these variables allow us to track multiple overflows to reach long durations like 1 or 2 seconds.

2. The Interrupt Service Routine (ISR) Logic

The ISR acts as a "switchboard" that triggers only when a timer "overflows" (hits its maximum value and rolls back to zero).

- **Timer0 Block (8-bit):**
  - **Polling:** It first checks if TMR0IF is set to ensure Timer0 caused the interrupt.
  - **Reloading:** It immediately reloads TMR0 with **61**. This ensures the timer only has to count 195 more ticks to overflow again, creating a precise **50ms** interval.
  - **Software Scaling:** Every time 50ms passes, the software counter increments. Once it hits **20**, a full second has passed (20times 50ms = 1000ms), and LED 1 on **RB0** is toggled.
  - **Flag Clearing:** The TMR0IF flag is manually cleared to 0 so the CPU can process future interrupts.
- **Timer1 Block (16-bit):**

- **Polling:** It checks TMR1IF (found in the PIR1 register).
- **Reloading:** Because Timer1 is 16-bit, the preload value (3036) is split into two 8-bit registers: TMR1H and TMR1L.
- **Long Duration Scaling:** Timer1 is configured with a 1:8 prescaler to count slowly. After the software counter hits **4**, a **2-second** interval is reached, toggling LED 3 on **RB2**.

3. Main Initialization Logic

Before the loop starts, the "gates" must be configured to allow hardware timing.

- **Port Setup:** Pins **RB0**, **RB2**, and **RD0** are set as outputs using the TRIS registers.
- **Timer0 Config:** Using OPTION_REG, the internal clock is selected, and the prescaler is set to **1:256** (the "gearbox" that slows down the ticks).
- **Timer1 Config:** Using T1CON, Timer1 is enabled with its own 1:8 prescaler.
- **Interrupt Enabling:**
  - TMR0IE and TMR1IE enable the individual timer "alarms".
  - **PEIE** (Peripheral Interrupt Enable) is set to 1 to allow Timer1 (a peripheral) to signal the CPU.
  - **GIE** (Global Interrupt Enable) is the master switch that allows any interrupt to function.

4. The Foreground Task (Main Loop)

- The while(1) loop contains a standard __delay_ms(100) routine to blink LED 2 on **RD0**.
- Even though this is a "blocking" delay that normally stops the CPU, the **hardware timers** continue counting independently in the background.
- The result is **Embedded Multi-tasking**: LED 2 blinks fast in the main loop while LED 1 and LED 3 blink at their precise rates in the background without interference.

Lab Questions & Analysis

1. Clock Frequency Sensitivity If you replace the 4 MHz crystal with a 20 MHz crystal without modifying the source code, the frequency of the Timer0 interrupts will **increase by a factor of 5**.

- **Mathematical Ratio:** Since the Instruction Clock (FCY) is F_OSC4, a 4 MHz crystal results in 1 MHz (1mu per tick), while a 20 MHz crystal results in 5 MHz (0.2per tick).
- **Effect:** The timer will count 5 times faster, meaning your 1-second LED toggle will now occur every **0.2 seconds**.

2. The Interrupt Flag Mechanism The instruction `INTCONbits.TMR0IF = 0;` is technically necessary because while the hardware sets the flag to 1 upon overflow, it cannot reset it.

- **Consequence of Failure:** If the software fails to clear this flag, the CPU will detect the flag as "set" immediately after exiting the ISR. This causes the microcontroller to re-enter the ISR infinitely, effectively "freezing" the execution of the main loop (the foreground task).

3. Embedded Multi-tasking

- **Definition:** Embedded Multi-tasking is the ability of a microcontroller to manage multiple independent tasks simultaneously by utilizing hardware peripherals (like timers) to handle time-critical events in the background.
- **Mechanism:** Hardware timers act as "silicon stopwatches" that run independently of the main CPU. This allows the CPU to execute code in the `while(1)` loop while the timers "ping" the CPU only when a specific interval is reached. This eliminates "busy-wait" software loops (like `delay_ms`), keeping the CPU free for other calculations.

4. Register Mapping & Documentation For Exercise 2, the following configurations were justified in the code:

- **OPTION_REG:** Configured `T0CS = 0` (Internal Instruction Clock) and `PSA = 0` with `PS2:PS0 = 111` to assign a 1:256 prescaler to Timer0.

2.2.2.2 OPTION_REG Register

The OPTION_REG Register is a readable and writable register, which contains various control bits to configure the TMR0 prescaler/WDT postscaler (single assignable register known also as the prescaler), the external INT interrupt, TMR0 and the weak pull-ups on PORTB.

Note: To achieve a 1:1 prescaler assignment for the TMR0 register, assign the prescaler to the Watchdog Timer.

REGISTER 2-2: OPTION_REG REGISTER (ADDRESS 81h, 181h)

R/W-1	R/W-1	R/W-1	R/W-1	R/W-1	R/W-1	R/W-1	R/W-1
$\overline{\text{RBPU}}$	INTEDG	T0CS	T0SE	PSA	PS2	PS1	PS0
bit 7							bit 0

bit 7 **$\overline{\text{RBPU}}$:** PORTB Pull-up Enable bit
1 = PORTB pull-ups are disabled
0 = PORTB pull-ups are enabled by individual port latch values

bit 6 **INTEDG**: Interrupt Edge Select bit
1 = Interrupt on rising edge of RB0/INT pin
0 = Interrupt on falling edge of RB0/INT pin

bit 5 **T0CS**: TMR0 Clock Source Select bit
1 = Transition on RA4/T0CKI pin
0 = Internal instruction cycle clock (CLKO)

bit 4 **T0SE**: TMR0 Source Edge Select bit
1 = Increment on high-to-low transition on RA4/T0CKI pin
0 = Increment on low-to-high transition on RA4/T0CKI pin

bit 3 **PSA**: Prescaler Assignment bit
1 = Prescaler is assigned to the WDT
0 = Prescaler is assigned to the Timer0 module

bit 2-0 **PS2:PS0**: Prescaler Rate Select bits

Bit Value	TMR0 Rate	WDT Rate
000	1 : 2	1 : 1
001	1 : 4	1 : 2
010	1 : 8	1 : 4
011	1 : 16	1 : 8
100	1 : 32	1 : 16
101	1 : 64	1 : 32
110	1 : 128	1 : 64
111	1 : 256	1 : 128

Legend:
R = Readable bit W = Writable bit U = Unimplemented bit, read as '0'
- n = Value at POR '1' = Bit is set '0' = Bit is cleared x = Bit is unknown

Note: When using Low-Voltage ICSP Programming (LVP) and the pull-ups on PORTB are enabled, bit 3 in the TRISB register must be cleared to disable the pull-up on RB3 and ensure the proper operation of the device

- **INTCON:** Set `GIE = 1` (Global Enable), `PEIE = 1` (Peripheral Enable for Timer1), and `TMR0IE = 1` (Timer0 Enable).

2.2.2.3 INTCON Register

The INTCON register is a readable and writable register, which contains various enable and flag bits for the TMR0 register overflow, RB port change and external RB0/INT pin interrupts.

> **Note:** Interrupt flag bits are set when an interrupt condition occurs regardless of the state of its corresponding enable bit or the global enable bit, GIE (INTCON<7>). User software should ensure the appropriate interrupt flag bits are clear prior to enabling an interrupt.

REGISTER 2-3: INTCON REGISTER (ADDRESS 0Bh, 8Bh, 10Bh, 18Bh)

R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-x
GIE	PEIE	TMR0IE	INTE	RBIE	TMR0IF	INTF	RBIF
bit 7							bit 0

bit 7 **GIE:** Global Interrupt Enable bit
- 1 = Enables all unmasked interrupts
- 0 = Disables all interrupts

bit 6 **PEIE**: Peripheral Interrupt Enable bit
- 1 = Enables all unmasked peripheral interrupts
- 0 = Disables all peripheral interrupts

bit 5 **TMR0IE**: TMR0 Overflow Interrupt Enable bit
- 1 = Enables the TMR0 interrupt
- 0 = Disables the TMR0 interrupt

bit 4 **INTE**: RB0/INT External Interrupt Enable bit
- 1 = Enables the RB0/INT external interrupt
- 0 = Disables the RB0/INT external interrupt

bit 3 **RBIE**: RB Port Change Interrupt Enable bit
- 1 = Enables the RB port change interrupt
- 0 = Disables the RB port change interrupt

bit 2 **TMR0IF**: TMR0 Overflow Interrupt Flag bit
- 1 = TMR0 register has overflowed (must be cleared in software)
- 0 = TMR0 register did not overflow

bit 1 **INTF**: RB0/INT External Interrupt Flag bit
- 1 = The RB0/INT external interrupt occurred (must be cleared in software)
- 0 = The RB0/INT external interrupt did not occur

bit 0 **RBIF**: RB Port Change Interrupt Flag bit
- 1 = At least one of the RB7:RB4 pins changed state; a mismatch condition will continue to set the bit. Reading PORTB will end the mismatch condition and allow the bit to be cleared (must be cleared in software).
- 0 = None of the RB7:RB4 pins have changed state

Legend:			
R = Readable bit	W = Writable bit	U = Unimplemented bit, read as '0'	
- n = Value at POR	'1' = Bit is set	'0' = Bit is cleared	x = Bit is unknown

- **T1CON:** Configured `T1CKPS = 11` for a 1:8 prescaler and `TMR1ON = 1` to start the 16-bit counter.

REGISTER 6-1: T1CON: TIMER1 CONTROL REGISTER (ADDRESS 10h)

U-0	U-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0
—	—	T1CKPS1	T1CKPS0	T1OSCEN	$\overline{\text{T1SYNC}}$	TMR1CS	TMR1ON
bit 7							bit 0

bit 7-6 **Unimplemented:** Read as '0'

bit 5-4 **T1CKPS1:T1CKPS0:** Timer1 Input Clock Prescale Select bits
11 = 1:8 prescale value
10 = 1:4 prescale value
01 = 1:2 prescale value
00 = 1:1 prescale value

bit 3 **T1OSCEN**: Timer1 Oscillator Enable Control bit
1 = Oscillator is enabled
0 = Oscillator is shut-off (the oscillator inverter is turned off to eliminate power drain)

bit 2 **$\overline{\text{T1SYNC}}$**: Timer1 External Clock Input Synchronization Control bit
When TMR1CS = 1:
1 = Do not synchronize external clock input
0 = Synchronize external clock input
When TMR1CS = 0:
This bit is ignored. Timer1 uses the internal clock when TMR1CS = 0.

bit 1 **TMR1CS**: Timer1 Clock Source Select bit
1 = External clock from pin RC0/T1OSO/T1CKI (on the rising edge)
0 = Internal clock (Fosc/4)

bit 0 **TMR1ON**: Timer1 On bit
1 = Enables Timer1
0 = Stops Timer1

Legend:			
R = Readable bit	W = Writable bit	U = Unimplemented bit, read as '0'	
- n = Value at POR	'1' = Bit is set	'0' = Bit is cleared	x = Bit is unknown

- **PIE1:** Set `TMR1IE = 1` to enable the specific interrupt for the Timer1 module.

REGISTER 2-4: PIE1 REGISTER (ADDRESS 8Ch)

R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0
PSPIE(1)	ADIE	RCIE	TXIE	SSPIE	CCP1IE	TMR2IE	TMR1IE
bit 7							bit 0

bit 7 **PSPIE:** Parallel Slave Port Read/Write Interrupt Enable bit(1)
1 = Enables the PSP read/write interrupt
0 = Disables the PSP read/write interrupt
Note 1: PSPIE is reserved on PIC16F873A/876A devices; always maintain this bit clear.

bit 6 **ADIE**: A/D Converter Interrupt Enable bit
1 = Enables the A/D converter interrupt
0 = Disables the A/D converter interrupt

bit 5 **RCIE**: USART Receive Interrupt Enable bit
1 = Enables the USART receive interrupt
0 = Disables the USART receive interrupt

bit 4 **TXIE**: USART Transmit Interrupt Enable bit
1 = Enables the USART transmit interrupt
0 = Disables the USART transmit interrupt

bit 3 **SSPIE**: Synchronous Serial Port Interrupt Enable bit
1 = Enables the SSP interrupt
0 = Disables the SSP interrupt

bit 2 **CCP1IE**: CCP1 Interrupt Enable bit
1 = Enables the CCP1 interrupt
0 = Disables the CCP1 interrupt

bit 1 **TMR2IE**: TMR2 to PR2 Match Interrupt Enable bit
1 = Enables the TMR2 to PR2 match interrupt
0 = Disables the TMR2 to PR2 match interrupt

bit 0 **TMR1IE**: TMR1 Overflow Interrupt Enable bit
1 = Enables the TMR1 overflow interrupt
0 = Disables the TMR1 overflow interrupt

Legend:			
R = Readable bit	W = Writable bit	U = Unimplemented bit, read as '0'	
- n = Value at POR	'1' = Bit is set	'0' = Bit is cleared	x = Bit is unknown

-

5. Signal Temporal Analysis

- **RB0 (LED 1):** Period = 1s, Frequency = 1Hz. Correlates to the Timer0 50ms interrupt triggering a toggle every 20 counts.
- **RD0 (LED 2):** Period = 0.2s, Frequency = 5Hz. Correlates to the `delay_ms(100)` high and `delay_ms(100)` low in the main loop.
- **RB2 (LED 3):** Period = 2s, Frequency = 0.5Hz. Correlates to the Timer1 500ms preload (3036) triggering a toggle every 4 counts.

Conclusion:

The successful completion of this lab demonstrates the transition from restrictive, blocking software delays to a sophisticated **Embedded Multi-tasking** architecture. By offloading timing responsibilities to independent hardware peripherals—the **8-bit Timer0** and **16-bit Timer1**—the CPU was freed from executing millions of redundant "NOP" instructions, allowing it to remain responsive to foreground tasks while managing precise background events.

Key technical takeaways include:

- **Hardware Independence:** Hardware timers act as "silicon stopwatches" that run regardless of the main program's state, preventing the CPU from becoming "blind" during time intervals.
- **Resolution and Scaling:** We leveraged the **8-bit resolution** of Timer0 for high-speed ticks and the **16-bit resolution** of Timer1 for longer-duration, 2-second intervals.
- **The "Switchboard" ISR:** A critical design principle was established by using a multi-vector Interrupt Service Routine that polls individual flags (`TMR0IF` and `TMR1IF`) without using `if-else` blocks, ensuring that all pending interrupts are processed with minimal latency.
- **Precision Maintenance:** We verified that manual register reloading (e.g., preloading **61** for Timer0) is essential every time an interrupt occurs to maintain consistent, instruction-perfect timing intervals.

Ultimately, this experiment confirms that hardware-timed interrupts are fundamental to professional embedded systems, enabling complex task scheduling—such as sensor sampling or display refreshing—without stalling the primary application logic.